Politecnico di Milano

Ingegneria del Software – a.a. 2006/07

Appello del 23 luglio 2007

Cognome

Nome

Matricola

Sezione (segnarne una) Baresi □, Ghezzi □, Morzenti □, SanPietro □

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.
6. Punteggio totale a disposizione: 25/30. La sufficienza si raggiunge con 16 punti, a cui sommare il risultato delle prove di laboratorio.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Totale .....

Esercizio 1 (punti 8)

Il seguente metodo statico

public static int rigaSommaMax(int [][] m) {... }

accetta come ingresso un array a due dimensioni m di cui si assume che rappresenti una matrice rettangolare (cioè tutte le righe hanno lo stesso numero di elementi; si noti che una matrice quadrata, in cui il numero delle righe è uguale a quello delle colonne, è considerato un caso particolare di matrice rettangolare, quindi accettabile come parametro) avente almeno due righe e due colonne, e produce come risultato l'indice della riga che i cui elementi hanno somma massima (se più righe hanno lo stesso valore massimo il metodo restituisce l'indice di una qualsiasi di queste).

a) Si scriva una specifica, in JML del metodo indicando la pre- e la post-condizione.

Soluzione

```
//@requires m!= null &&
// la matrice ha almeno due righe
//@ m.length>1 &&
// tutte le righe hanno almeno due elementi
//@ (\forall int i; 0<=i<m.length; m[i]!=null && m[i].length>1) &&
//tutte le righe hanno la stessa lunghezza
//@ (\forall int i; 0<=i<m.length; (\forall int j; i<j<m.length; m[i].length == m[j].length) )

//@ensures
// non esiste una riga per la quale la somma degli elementi sua maggiore che per la riga di indice \result
//@ !(\exists int i; 0<=i<m.length;
//@   (\sum int j; 0<=j<m[i].length; m[i][j]) > (\sum int j; 0<=j<m[\result].length; m[\result][j])
```

b) si trasformi la specifica precedente facendo in modo che il metodo accetti in ingresso una qualsiasi "matrice" con almeno due righe e due colonne, ma se la matrice non è rettangolare (cioè le righe non sono tutte della stessa lunghezza) esso lanci un'eccezione *MatriceNonRettangolareException* (tutti gli altri requisiti sono invariati).

Soluzione

```
//@requires m!= null &&
// la matrice ha almeno due righe
//@ m.length>1 &&
// tutte le righe hanno almeno due elementi
//@ (\forall int i; 0<=i<m.length; m[i].length>1) &&

//@ensures
//tutte le righe hanno la stessa lunghezza
(\forall int i; 0<=i<m.length; (\forall int j; i<j<m.length; m[i].length == m[j].length) ) &&
// non esiste una riga per la quale la somma degli elementi sia maggiore che per la riga di indice
\result
!(\exists int i; 0<=i<m.length;
  (\sum int j; 0<=j<m[i].length; m[i][j]) > (\sum int j; 0<=j<m[\result].length; m[\result][j])
//@signals (MatriceNonRettangolareException e)
//@ (! (\forall int i; 0<=i<m.length; (\forall int j; i<j<m.length; m[i].length == m[j].length)   ) )
```

c) Si indichi quale clausola deve essere *aggiunta* alla post-condizione del punto a) per specificare che, nel caso in cui la matrice abbia più righe con lo stesso valore massimo della somma degli elementi, viene restituito come risultato l'indice della prima tra queste (quella che ha cioè indice minimo).

Soluzione: occorre aggiungere la clausola

```
&& !(\exists int k; k<\result;
        (\sum int j; 0<=j<m[k].length; m[k][j]) ==
          (\sum int j; 0<=j<m[\result].length; m[\result][j]))
```

Esercizio 2 (punti 13)

Si consideri un sistema di gestione del traffico telefonico. Il sistema, fra le altre cose, tiene traccia degli utenti e delle loro chiamate. Una chiamata ha una data, un'ora, una durata e un numero. Le chiamate possono essere vocali, e in tal caso hanno anche uno scatto alla risposta e un costo al secondo, o di tipo dati, che invece hanno un volume di dati scambiati (in Kbyte) e un costo in volume (ossia al Kbyte).
Formalmente, le tre classi sono:

```
abstract public class Chiamata {
        abstract public Data data();
        abstract public int ora(); //HHMMSS
        abstract public int durataInSecondi();
        abstract public int costoTotale();
        abstract public String numeroChiamato();
}

public class ChiamataVocale extends Chiamata {
        public int costoScatto();
        public int costoAlSecondo();
}

public class ChiamataDati extends Chiamata {
        public int costoKB();
        public int volumeKByte();
}
```

La specifica della classe Data è:

```
public classe Data {
        public int giorno();
        public int mese();
        public int anno();
        //@ ensures (*\result è true sse this precede strettamente d *)
        public boolean precede(Data d);
}
```

Per ogni utente, il sistema tiene traccia del nome, del credito residuo e delle chiamate effettuate. E' inoltre possibile effettuare una ricarica di un numero intero di euro.
L'interfaccia della classe è così definita:

```
public class Utente {
//@ensures (* \result è il credito in centesimi di euro ancora disponibile *)
public int creditoResiduo();
//@ensures ....
```

```
public void ricarica(int euro);
//@ensures (*\result è un generatore, in ordine cronologico, delle chiamate effettuate alla data d*)
public Iterator<Chiamata>  chiamateInData(Data d);
}
```

a) Scrivere in JML la postcondizione del metodo ricarica.

SOLUZIONE: this.creditoResiduo()== 100*euro + \old(this.creditoResiduo());

Sia dato il seguente rep per la classe Utente:

```
private ArrayList<Chiamata> log;
//contiene tutte le chiamate effettuate in ordine cronologico
int spesaTotaleAnnoCorrente; //la spesa complessiva in centesimi di euro dell'utente nell'anno ) corrente fino alla data attuale
Data dataCorrente; //la data attuale
int totaleRicariche; //il credito complessivo, in euro, acquistato dall'utente
int creditoResiduo; //il credito ancora disponibile
```

b) Scrivere l'invariante di rappresentazione che stabilisce la correttezza e la mutua consistenza dei dati che costituiscono il rep.

SOLUZIONE:

log in ordine crescente && ultima chiamata del log in data <=dataCorrente && totaleRicariche è uguale al totale dei costi delle chiamate sommato al credito residuo && spesaTotaleAnno è pari al costo complessivo delle chiamate per il solo anno corrente.

```
log !=null && dataCorrente!=null && (\forall int j; 0<= j && j<log.size(); log.get(j) !=null) &&
 (\forall int j; 0<= j && j<log.size()-1;
                    (log.get(j).data().precede(log.get(j+1).data()) ||
                     log.get(j).data().equals(log.get(j+1).data()) &&
                            log.get(j).ora()<log.get(j+1).ora()) &&
( log.get(log.size()-1).data().precede(dataCorrente) ||
    log.get(log.size()-1).data().equals(dataCorrente) )
totaleRicariche == 100*(\sum int j; 0<= j && j<log.size(); log.get(j).costoTotale())+creditoResiduo
&&
spesaTotaleAnno ==
   (\sum int j; 0<= j && j<log.size()&&   log.get(j).data().anno()==dataCorrente.anno();
          log.get(j).costoTotale())
```

c) Con riferimento al rep introdotto al punto precedente, implementare il metodo iteratore chiamateInData(), includendo anche la definizione di tutte le classi appropriate.

SOLUZIONE:

```
public Iterator<Chiamata>  chiamateInData(Data d) {return new UtenteGen(this,d);}
private static class UtenteGen implements Iterator<Chiamata> {
        private Utente p; //l'Utente sul cui log si vuole iterare
        private int n; //posizione del prox el. da considerare
        private Data dataAttuale;
        //@requires u!=null;
        UtenteGen(Utente u,Data d) {
                dataAttuale=d;
                p=u;
                for(int i; 0<=i && i<p.log.size() && p.log.get(i).data().precede(dataAttuale);i++);
                n=i;
        }
        public boolean hasNext ( ) {
                return n<p.log.size() && dataAttuale.equals(p.log.get(n).data());
        }
        public int next ( ) throws NoSuchElementException {
                if (!hasNext()) throw new NoSuchElementException();
                return p.log.get(n++);
        }
}
```

Esercizio 3 (punti 4)
Si consideri la seguente variante della classe Utente introdotta nell'esercizio 2, con parte dell'interfaccia definita come segue.

```
public class Utente {
        ....
        //@ensures (*\result è la somma del costo industriale di tutte le chiamate effettuate
        //@ alla data d*)
        public int costoIndustrialeChiamateInData(Data d);

        //@ensures \result > costoIndustrialeChiamateInData(d) / 2 &&
        //@                 \result <= costoIndustrialeChiamateInData(d) * 2
        public int importoFatturabileInData(Data d);
}
```

Vengono inoltre definite le due classi UtenteAgevolato e UtentePrivilegiato, possibili classi eredi di Utente, che usufruiscono di tariffe particolarmente favorevoli.

```
public class UtenteAgevolato  {
        ....
        //@ensures \result == costoIndustrialeChiamateInData(d) * 0.7
        public int importoFatturabileInData(Data d);
}

public class UtentePrivilegiato  {
        ....
        public final int TETTO = ...;
        //@ensures \result ==
        //@ costoIndustrialeChiamateInData(d) * 1.5 <= TETTO ?
        //@                 costoIndustrialeChiamateInData(d) * 1.5  : TETTO
        public int importoFatturabileInData(Data d);
}
```

Le classi UtenteAgevolato e UtentePrivilegiato sono definibili come classi eredi della classe Utente senza violare il principio di sostituzione? Motivare adeguatamente la risposta, nel caso positivo argomentando che sono soddisfatte le regole di sostituibilita', oppure nel caso negativo mostrando che non lo sono mediante un controesempio.

Soluzione

La classe UtenteAgevolato soddisfa il principio di sostituzione, perchè la postcondizione

\result == costoIndustrialeChiamateInData(d) * 0.7

Implica logicamente la postcondizione

\result > costoIndustrialeChiamateInData(d) / 2 &&
        \result < costoIndustrialeChiamateInData(d) * 2

Invece la classe UtentePrivilegiato non lo soddisfa, perchè l'imposizione del TETTO all'importo fatturabile può far sì che venga violata la cobndizione di fatturare almeno metà del costo industriale; in particolare, per valori di costoIndustrialeChiamateInData(d) sufficientemente elevati, si possono verificare entrambe le seguenti condizioni

costoIndustrialeChiamateInData(d) * 1.5 > TETTO

che fa si che \result == TETTO, e

TETTO < costoIndustrialeChiamateInData(d)/2

violando quindi la postcondizione del metodo importoFatturabileInData che prescrive che \result >= costoIndustrialeChiamateInData(d)/2